Introduction to Python

In this lab, we will cover some basic Python syntax and provide snippets that you can use in your code. Familiarity with another programming language may be helpful, since knowledge of basic programming concepts is assumed. No prior Python experience is required. Interacting with Python is one of the best ways to learn it – try out https://colab.research.google.com/ using the "new notebook" button to set up a notebook for running the code. Let's get started!

Variables and Printing

A variable is a way to store and manipulate data when programming. We declare a variable by giving it a name and a value. Values of the variable can change over time, and can be different types, such as integers or strings.

Here is an example of declaring a variable:

```python
y = 3.45
```

Here the name of the variable is `*y*` and the value is `*3.45*`.

To print the value of a variable, we pass the variable to a special function called `print`:

```python
x = "Hello World"
print(x)
```

Read the below snippets:

```python
# Integer
num = 5
print(num)

# Boolean
var = True
print(var)

# String
```

```python
hi = "Hello World"
print(hi)

# Double (decimal or float)
dec = 3.3
print(dec)
```

Python

```python
# Adding and combining variables

total = num + 2
print(total)

difference = num - 2
print(difference)

product = num * 2
print(difference)

square = num ** 2  # Sets the variable square to equal the value of num^2
print(square)

remainder = num % 2  # Remainder when num is divided by 2
print(remainder)
```

Python

```python
# F-strings

# One popular way to create strings that have variables in them is to use an
f-string expression.
# All code inside of curly brackets is executed as normal python code, then the
output is converted to a string.

print(f"The integer num has the value {num}")
print(f"We can explicitly convert num to a string, too: {str(num)}")
```

Python

```python
# Converting types
```

```
print(f"2 + 2 = {int('2') + 2}")
print(f"When we use floats, 2 + 2 = {float('2') + 2}")
print(
    f"If we want to convert integers to booleans, we see that 1 is actually
{bool(1)} and 0 is {bool(0)}"
)
```

Strings

A string is just another name for a piece of text. Python provides lots of built-in functions for manipulating string variables. To declare a string variable, you must wrap the string in single or double quotations:

Python
```
my_str = 'This is a string'
double_quote_str = "This is a string wrapped in double quotes"
```

In the above examples, your strings cannot include line-breaks (or else you will get an error). We can write multi-line strings in Python using three sets of single quotes:

Python
```
long_str = '''
      This string is taking up
      multiple lines.
'''
```

Read the following code snippets. You'll see several functions we can use to manipulate strings.

Python
```
# Compute the length, or number of characters in a string

s = "Hi my name is Shreya. I like mac and cheese."
print(len(s))
```

```python
# Split the string into an array or list based on a delimiter

print(s.split(" "))  # Delimiter is a space
print(s.split("."))  # Delimiter is a period
```

```python
# Convert string to lower or uppercase

print(s.lower())
print(s.upper())
```

```python
# Replace occurrences of a character

print(s.replace("e", "3"))  # Replaces the character e with 3
```

```python
# Strip whitespace

ws = "   Hey what's up  "
print(ws)
print(ws.strip())
```

More string functions can be found here.

Lists

We can store lists of information in Python. Python lists can contain elements with different data types, like:

```python
my_list = ["Nick", "Shreya", 3.0, 10]
```

To create an empty list, both of the following statements are equivalent:

Python
```
emptyList1 = list()
emptyList2 = []
```

Python
```
# Useful functions for lists of numbers

nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print(max(nums))
print(min(nums))
print(sum(nums))
```

Python
```
# Obtain specific elements of a list ("index" into a list with square brackets)
and compute the length of a list (number of elements)

bootcamp_instructors = ["Shreya", "Nitya", "Reese", "Nick"]
print("The element at the 1st position is: " + bootcamp_instructors[0])
print("The element at the 2nd position is: " + bootcamp_instructors[1])
print("The element at the last position is: " + bootcamp_instructors[-1])
print("The element at the second-to-last position is: " +
bootcamp_instructors[-2])
```

Python
```
# You can also obtain a "slice" of your list by using colons inside of the
indexing syntax.
# A slice returns a sequence of items, with list[start:stop] returning all
items from
# start to stop - 1

print("The first two instructors are: " + str(bootcamp_instructors[0:2]))

# You can omit either start or stop, to have the slice start at the beginning
(or end at the end) of your list:

print(
```

```
    f"The list of instructors from the second element onwards is:
{bootcamp_instructors[1:3]}"
)
print(
    f"The list of instructors without the last element is:
{bootcamp_instructors[:-1]}"
)
print(f"The middle 2 instructors are: {bootcamp_instructors[1:3]}")

# or omit both both:
print(f"All of the bootcamp instructors are {bootcamp_instructors[:]}")

print(f"There are {len(bootcamp_instructors)} bootcamp instructors.")
```

Python

```
# List operations and methods

print(bootcamp_instructors + bootcamp_instructors)
print(bootcamp_instructors * 2)

# Find index of a specific element
print(bootcamp_instructors.index("Shreya"))

# Add element to list
bootcamp_instructors.append("Obama")
print(bootcamp_instructors)

# Remove element from list
bootcamp_instructors.remove("Obama")
print(bootcamp_instructors)

# Add element at specific position to list
bootcamp_instructors.insert(0, "Obama")
print(bootcamp_instructors)

# Remove element at specific position in list
del bootcamp_instructors[0]
print(bootcamp_instructors)

# Reverse list
bootcamp_instructors.reverse()
print(bootcamp_instructors)
```

```
# Sort list
bootcamp_instructors.sort()
print(bootcamp_instructors)
```

Dictionaries

A dictionary contains keys and values, where each key in the dictionary has a specific value. Values can be any type, such as strings or lists. Keys can be any type except a list.

An example of a dictionary is below:

Python
```
years = {
    "freshman": 9,
    "sophomore": 10,
    "junior": 11,
    "senior": 12,
}
```

To get the value of a key in the dictionary, you can "index" into the dictionary:

Python
```
years["freshman"] # returns 9
```

The following snippets are examples of dictionary functions.

Python
```
# Create a new dictionary and add elements to it

my_dict = {"freshman": 9, "sophomore": 10}

my_dict["junior"] = 11
my_dict["senior"] = 12
```

```
print(my_dict)

# Useful functions
print("The keys in the dictionary are " + str(my_dict.keys()))
print("The values in the dictionary are " + str(my_dict.values()))
print("The items in the dictionary are " + str(my_dict.items()))
```

For loops

To iterate over lists or dictionaries, we need to use for loops:

```python
for i in range(0, 10):
  print(i)
```

If you want to print every two numbers, you can modify the 'range' function:

```python
for i in range(0, 10, 2):
  print(i)
```

The following snippets are examples of how we can use for loops with different data structures.

```python
# For loops with lists

bootcamp_instructors = ["Shreya", "Nitya", "Nick", "Reese"]

# For-each loop (simplest for loop)
print("The bootcamp instructors are: ")
for instructor in bootcamp_instructors:
    print(instructor)

# For loop with indices
```

```
print("\nThe bootcamp instructors are: ")
for idx, instructor in enumerate(bootcamp_instructors):
    print(instructor + " is at index " + str(idx))

# For loop only with indices
print("\nThe bootcamp instructors are: ")
for i in range(len(bootcamp_instructors)):
    print(bootcamp_instructors[i])
```

Python

```
# For loops with dictionaries

years = {"freshman": 9, "sophomore": 10, "junior": 11, "senior": 12}

# Iterate through key, value pairs

for name, yr in years.items():
    print(name + " corresponds to year " + str(yr))

# Iterate through keys
print("\n")
for name in years.keys():
    print(name + " corresponds to year " + str(years[name]))
```

If statements

The syntax for an if and else if statement is:

Python

```
if (....):
  # do something
elif (....):
  # do something else
else:
  # do something else
```

Functions

We often want to use code we've written multiple times in different places. The easiest way to do this is with functions. A function is just a way to name a particular snippet of code that takes in some variables returns a value. For example, the following function returns the fourth power of the number we pass in.

```python
def fourth_power(x):
  y = x * x * x * x
  return y
```

The `def` keyword indicates that we are defining a function. We always follow `def` with the name of the function. After the name, we add a pair of parentheses, and inside we put a comma-separated list of inputs (called arguments). Inside the function (which we indicate by indenting), we write our code. The special `return` keyword will immediately end the function and return whatever is following it.

As another example, let's say we want a function that searches in a list for a particular variable, and returns `True` if the variable is in the list and `False` otherwise:

```python
def find_in_list(l, s):
  for item in l:
    if item == s:
      return True
  return False
```

Once we've define a function, we can call the function (i.e., make the code inside it run) by writing the name along with some values or variables for the arguments:

```python
my_list = ["banana", "apple", "orange"]
found = find_in_list (my_list, "apple")
print (found)
```

The above code will print out `True`. Below we show some more examples of functions.

Python

```python
# Let's make a function that just prints out the string "Hello, World!"


def print_hello_world():
    print("Hello, World!")


print_hello_world()
```

Python

```python
# Let's make a function that creates a string created by concatenating the
numbers 1, 2, 3, ....
# Example: a string that goes up to 7 would be "1234567".
# Example: a string that goes up to 11 would be "1234567891011".


def create_number_string(n):
    final = ""
    for i in range(1, n + 1):
        final += str(i)
    return final


print("We can do it up to 7: '" + create_number_string(7) + "'")
print("We can do it up to 11: '" + create_number_string(11) + "'")
```